LAPORAN PRAKTIKUM STRUKTUR DATA

"Implementasi Abstract Data Type (ADT) Jam"

Disusun Oleh:

Aryahiyahul Fikra
2511532026

Dosen Pengampu:

Dr. Wahyudi S.T, M.T

Asisten Pembimbing:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI 2
1.1 Latar Belakang 3
1.2 Rumusan Masalah 3
1.3 Tujuan Praktikum 3
BAB II 4
PEMBAHASAN 4
2.1 Konsep Dasar Program 4
2.2 Struktur Class Jam_2511532026 4
2.3 Validasi Data 5
2.4 Getter dan Setter 5
2.5 Konversi Waktu 6
2.6 Operasi Perbandingan 6
2.7 Operasi Aritmatika Waktu 7
2.8 Operasi Increment & Decrement 7
2.9 Perhitungan Durasi 8
2.10 Format Output 8
2.11 Analisis Class JamDriver_2511532026 9
2.12 Analisis Class JamDriver2_2511532026 9
BAB III 11
PENUTUP 11
3.1 Kesimpulan 11

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman berorientasi objek (OOP), konsep *Abstract Data Type (ADT)* digunakan untuk merepresentasikan suatu objek beserta operasi yang dapat dilakukan terhadapnya. Salah satu contoh implementasi ADT adalah objek waktu (jam), yang terdiri dari jam, menit, dan detik.

Pada praktikum ini, dibuat sebuah class Jam_2511532026 untuk merepresentasikan waktu, serta dua class driver yaitu JamDriver_2511532026 dan JamDriver2_2511532026 untuk menguji berbagai operasi yang tersedia pada objek jam.

Dengan adanya program ini, diharapkan mahasiswa dapat memahami konsep:

1. Enkapsulasi
2. Method (getter, setter, dan operasi)
3. Operasi aritmatika pada objek
4. Penggunaan class driver

1.2 Rumusan Masalah

Adapun rumusan masalah dalam praktikum ini adalah:

1. Bagaimana cara membuat ADT Jam menggunakan Java?
2. Bagaimana mengimplementasikan operasi-operasi pada objek waktu?
3. Bagaimana cara membandingkan dan menghitung durasi antar waktu?
4. Bagaimana cara membuat program driver untuk menguji class?

1.3 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Membuat class Jam dengan atribut jam, menit, dan detik
2. Mengimplementasikan berbagai operasi pada objek jam
3. Menggunakan method seperti toSecond, compareTo, dan lainnya
4. Membuat driver program untuk pengujian

BAB II
PEMBAHASAN

2.1 Konsep Dasar Program

Program ini dibuat menggunakan konsep **Object-Oriented Programming (OOP)**, dengan tujuan merepresentasikan waktu sebagai sebuah objek.

Konsep OOP yang digunakan meliputi:

1. Enkapsulasi

Atribut hh, mm, dan ss dibuat private agar tidak bisa diakses langsung dari luar class.

```
private int hh;
private int mm;
private int ss;
```

Akses dilakukan melalui method (getter/setter).

2. Abstraksi

Class Jam_2511532026 menyederhanakan konsep waktu menjadi objek dengan operasi:

- Konversi
- Perbandingan
- Aritmatika waktu

3. Reusability

Method seperti toSecond() dan fromSeconds() digunakan ulang oleh banyak method lain.

```
ther) {return fromSeconds (this.toSecond() + other.toSecond());}
other) {return fromSeconds (Math.floorMod(this.toSecond() - other.toSecond(), 24*3600));}
fromSeconds(this.toSecond() + 1);}
eturn fromSeconds(this.toSecond()+ Math.max(0, n));}
fromSeconds(Math.floorMod(this.toSecond() - 1, 24*3600 ));}
eturn fromSeconds (Math.floorMod(this.toSecond() - Math.max(0, n), 24*3600));}
```

2.2 Struktur Class Jam_2511532026

Class ini terdiri dari beberapa bagian utama:

1. Atribut

Atribut	Tipe	Fungsi
hh	int	Menyimpan jam
mm	int	Menyimpan menit

Atribut	Tipe	Fungsi
ss	int	Menyimpan detik

2. Constructor

public Jam_2511532026(int h, int m, int s)

```
public Jam_2511532026( int h,int m,int s) {
    this.hh = h; this.mm = m; this.ss= s;
```

Fungsi:

- Menginisialisasi objek jam

2.3 Validasi Data

public static boolean isValid(int h, int m, int s)

```
public static boolean isValid(int h, int m, int s) {
    return (0<= h && h <=23) && (0 <= m && m <=59) && (0<= s && s <= 59);
```

Fungsi:

- Memastikan input berada dalam rentang:
  - Jam: 0–23
  - Menit: 0–59
  - Detik: 0–59

2.4 Getter dan Setter

Getter:

```
public int getHour() {return hh;}
public int getMinute() {return mm;}
public int getSecond() {return ss;}
```

Setter:

```
public void getHour (int h) {this.hh= h;}
public void getMinute (int m) {this.mm=m ;}
public void getSecond (int s) {this.ss= s;}
```

2.5 Konversi Waktu

a. toSecond()

```
public int toSecond() {return hh*3600 + mm * 60 +ss;}
public static Jam_2511532026 fromSeconds (int total) {
    if (total <0) throw new IllegalArgumentException("detik negatif");
    total %= 24 *3600;
    int h = total /3600; total %= 3600;
    int m = total / 60; int s = total %60;
    return new Jam_2511532026(h,m,s);
```

Mengubah waktu menjadi total detik:

detik = (jam × 3600) + (menit × 60) + detik

Contoh:

01:01:01 = 3600 + 60 + 1 = 3661 detik

b. fromSeconds()

```
public static Jam_2511532026 fromSeconds (int total) {
    if (total <0) throw new IllegalArgumentException("detik negatif");
    total %= 24 *3600;
    int h = total /3600; total %= 3600;
    int m = total / 60; int s = total %60;
    return new Jam_2511532026(h,m,s);
```

Fungsi:

Mengubah detik menjadi objek waktu.

Proses:

1. Normalisasi ke 24 jam
2. Hitung jam, menit, detik

2.6 Operasi Perbandingan

Method:

```
public int compareTo ( Jam_2511532026 other) {return Integer.compare(this.toSecond(), other.toSecond()); }
public boolean equals(Object o) {
    if (!(o instanceof Jam_2511532026 j))return false;
    return hh == j.hh && mm == j.mm && ss == j.ss;
```

Fungsi:

Membandingkan dua waktu berdasarkan detik.

Output:

Nilai	Arti
> 0	Lebih besar

Nilai	Arti
< 0	Lebih kecil
= 0	Sama

equals() dan hashCode()

```
 public boolean equals(Object o) {
     if (!(o instanceof Jam_2511532026 j))return false;
     return hh == j.hh && mm == j.mm && ss == j.ss;
 }
lic int hashCode () {return java.util.Objects.hash(hh,mm,ss);}
```

Digunakan untuk:

- Membandingkan objek
- Mendukung struktur data (HashSet, HashMap)

2.7 Operasi Aritmatika Waktu

1. Penjumlahan

plus(Jam_2511532026 other)

Menjumlahkan dua waktu:

total detik = detik1 + detik2

2. Pengurangan

minus(Jam_2511532026 other)

Menggunakan:

Math.floorMod()

Fungsi:

Menghindari hasil negatif

Tetap dalam rentang 24 jam

2.8 Operasi Increment & Decrement

1. nextSecond()

Menambah 1 detik

2. prevSecond()

Mengurangi 1 detik

3. nextNSecond(int n)

Menambah n detik

4. prevNSecond(int n)

Mengurangi n detik

Kelebihan:

Fleksibel

Mendukung operasi waktu real-time

```
public Jam_2511532026 plus(Jam_2511532026 other) {return fromSeconds (this.toSecond() + other.toSecond());}
public Jam_2511532026 minus(Jam_2511532026 other) {return fromSeconds (Math.floorMod(this.toSecond() - other.toSecond(), 24*3600));}
public Jam_2511532026 nextSecond() {return fromSeconds(this.toSecond() + 1);}
public Jam_2511532026 nextNSecond(int n) {return fromSeconds(this.toSecond()+ Math.max(0, n));}
public Jam_2511532026 prevSecond() {return fromSeconds(Math.floorMod(this.toSecond() - 1, 24*3600 ));}
public Jam_2511532026 prevNSecond(int n) {return fromSeconds (Math.floorMod(this.toSecond() - Math.max(0, n), 24*3600));}
```

2.9 Perhitungan Durasi

Method:

durasiDetik(Jam_2511532026 jaw, Jam_2511532026 jakh)

Fungsi:

Menghitung selisih dua waktu.

Catatan:

- Bisa menghasilkan nilai negatif
- Belum menangani kasus lintas hari secara sempurna

2.10 Format Output

public String toString()

Menggunakan:

String.format("%02d:%02d:%02d", hh, mm, ss);

Hasil:

- Selalu 2 digit
- Format standar: HH:MM:SS

Lampiran 2.9 dan 2.10:

```
public static int durasiDetik(Jam_2511532026 jaw, Jam_2511532026 jakh) {return jakh.toSecond() - jaw.toSecond();}

public String toString() {return String.format("%02d:%02d:%02d", hh,mm, ss);}
```

2.11 Analisis Class JamDriver_2511532026

```java
package pekan1_2511532026;

public class JamDriver_2511532026 {

    public static void main(String[] args) {
        Jam_2511532026 a = new Jam_2511532026(23,59,50);
        Jam_2511532026 b = new Jam_2511532026(0,0,15);
        System.out.println("a      =" +a);
        System.out.println("b      =" +b);
        System.out.println("a+b      =" +a.plus(b));
        System.out.println("next 20s  =" +a.nextNSecond(20));
        System.out.println("durasi (a,b)    =" +Jam_2511532026.durasiDetik(a,b));
        System.out.println("a.compareTo(b)   =" +a.compareTo(b));
```

Karakteristik:

- Data langsung ditentukan (hardcoded)
- Cocok untuk testing cepat

Operasi yang diuji:

- Penjumlahan
- Penambahan detik
- Perbandingan
- Durasi

2.12 Analisis Class JamDriver2_2511532026

```java
package pekan1_2511532026;
import java.util.*;
public class JamDriver2_2511532026 {

    public static void main(String[] args) {
        Scanner i = new Scanner(System.in);
        System.out.println ("=== Program Driver Objek Jam ===");

        System.out.println("\n--- Input Jam ---");
        Jam_2511532026 j1 = buatJamDariInput(i);

        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511532026 j2 = buatJamDariInput(i);

        System.out.println("\n--- hasil operasi ---");
        System.out.println("Jam 1 (String)        :" + j1.toString());
        System.out.println("Jam 2 (String)        :" + j2.toString());
        System.out.println("Jam 1 dalam detik       :" + j1.toSecond());
        System.out.println("Jam 2 dalam detik       :" + j2.toSecond());

        int perbadingan = j1.compareTo(j2);
        if(perbadingan > 0) {
            System.out.println("Status                  : Jam 1 lebih lambat (setelah) Jam 2");
        } else if (perbadingan < 0  ) {
            System.out.println("Status                  : Jam 1 lebih awal (setelah) Jam 2");
        }else {
            System.out.println("Status                  : Jam 1 dan Jam 2 sama persis");
        }

        // 5. Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2)           : " + Jam_2511532026.durasiDetik(j1, j2) + " detik" );

        Jam_2511532026 jNext = j1.nextSecond();
```

```java
        System.out.println("Jam 1 Detik Berikutnya      : " + jNext);

        Jam_2511532026 jPrev = j1.prevSecond();
        System.out.println("Jam 1 Detik Sebelumnya      : " + jPrev);

        //6. Operasi Penjumlahan Jam
        Jam_2511532026 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 = J2               : " + jHasilPlus);

        i.close();
        System.out.println("\nProgram Selesai.");
    }


    private static Jam_2511532026 buatJamDariInput(Scanner sc) {
        int h, m, s;
        while (true) {
            System.out.print("Masukkan Jam (0-23)       : ");
            h = sc.nextInt();
            System.out.print("Masukkan Menit (0-59)     : ");
            m = sc.nextInt();
            System.out.print("Masukkan Detik (0-59)     : ");
            s = sc.nextInt();

            // memanggil method isValid
            if (Jam_2511532026.isValid(h, m, s)) {
                return new Jam_2511532026(h, m, s);
            } else {
                System.out.println("[Error] Input tidak valid! Silakan ulangi.\n");
            }
        }
```

Karakteristik:

- Menggunakan input user (Scanner)
- Lebih interaktif

Fitur utama:

1. Input jam dengan validasi
2. Perbandingan dua waktu
3. Operasi aritmatika
4. Output lengkap

Fungsi Penting

buatJamDariInput()

Fungsi:

- Loop sampai input valid
- Menggunakan isValid()

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa pemrograman berorientasi objek (Object-Oriented Programming/OOP) memudahkan dalam memodelkan suatu permasalahan ke dalam bentuk objek yang terstruktur dan sistematis. Dengan menggunakan konsep seperti enkapsulasi dan penggunaan method, sebuah data dapat diolah dengan lebih aman dan efisien.
Selain itu, pembuatan Abstract Data Type (ADT) memungkinkan data dan operasi yang berkaitan dapat dikelompokkan dalam satu kesatuan. Hal ini membuat program menjadi lebih terorganisir, mudah dipahami, serta mudah dikembangkan di kemudian hari.
Melalui praktikum ini juga dapat dipahami bahwa proses validasi data, pengolahan input, serta pengujian program merupakan bagian penting dalam membangun aplikasi yang berjalan dengan baik dan sesuai dengan kebutuhan.